

System Driven Hardware Design

-Analog to Digital Conversion (ADC) -

International Master of Science

Prof. Dr.-Ing. Stephan Bannwarth

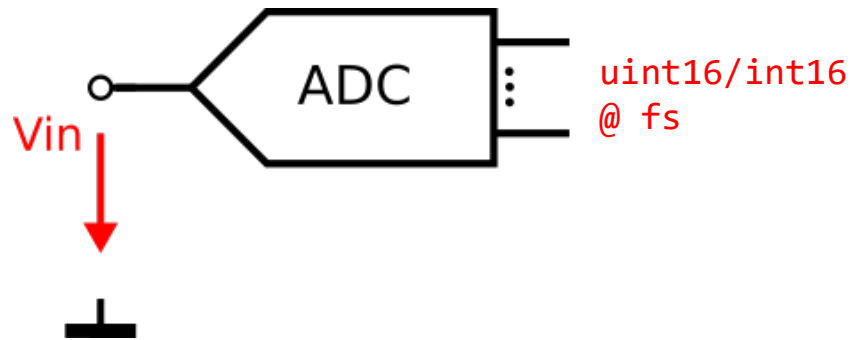
ADC: enables digital signal processing

Agenda

- o ADC
- o Exercise 1: ADC Reading

Analog: continues time and continues values

Digital: discrete time and discrete values



Discrete time

- Nyquistrate (f_s)

Discrete value

- Number of Bits
- Noise and Distortion (SINAD)
- Resolution (Vlsb)

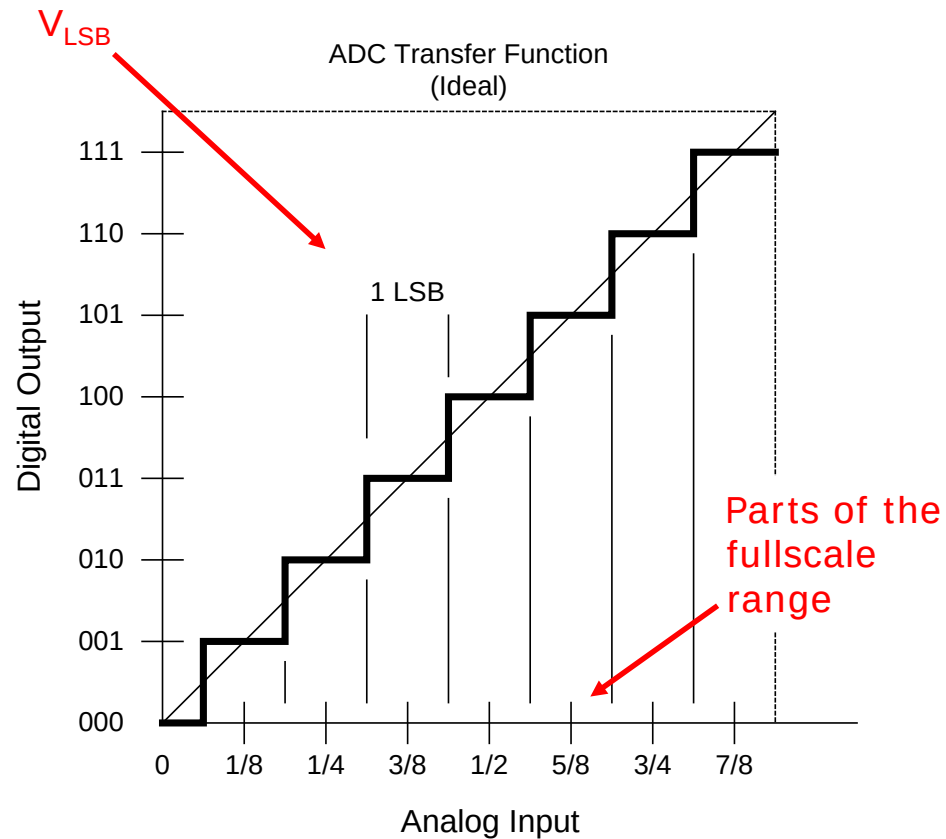
Circuit design

- Single ended
- Differential

Signal Processing in C

- Signed/ Unsigned integer
- Resolution and bit size

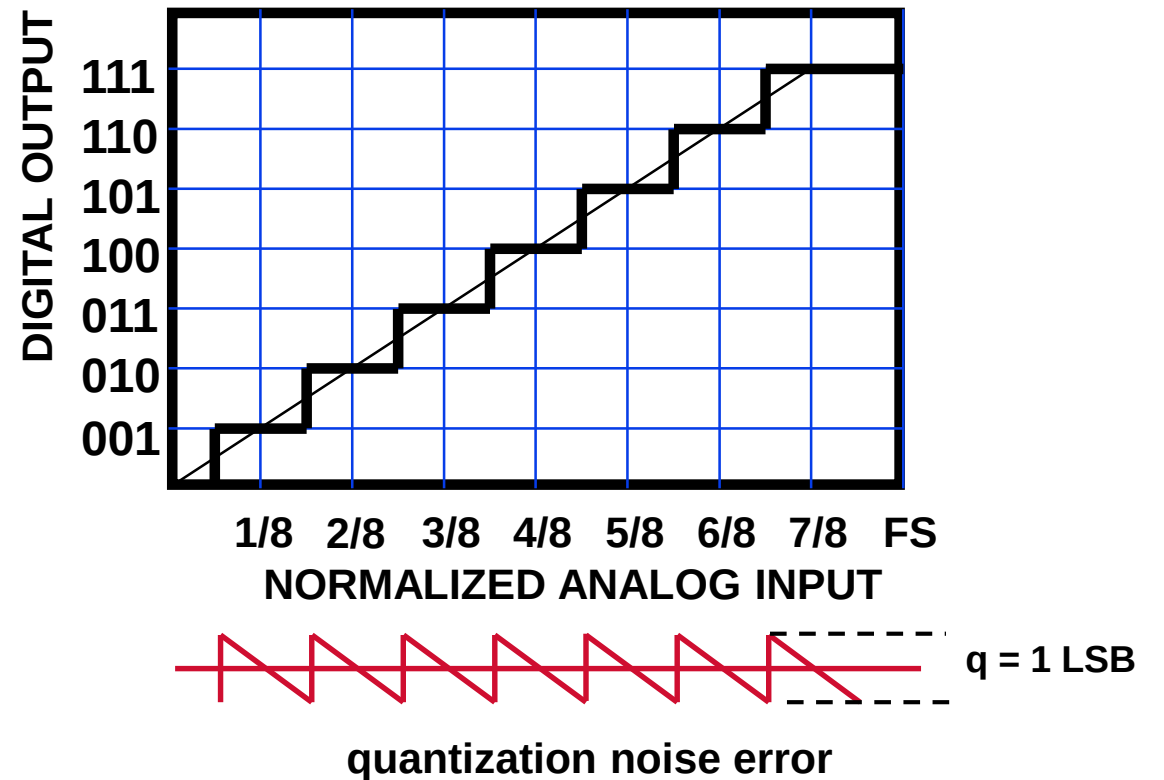
Ideal 3-Bit „Staircase“ ADC



Least Significant Bit and Fullscale Range

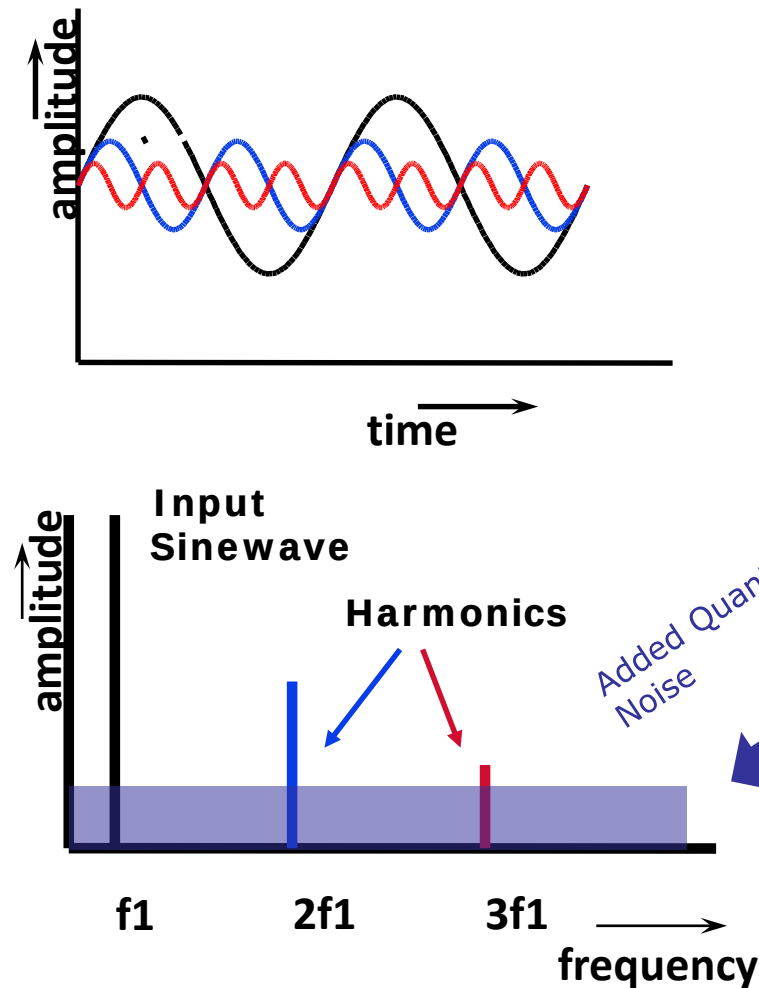
$$V_{LSB} = \frac{V_{FS}}{2^n}$$

Quantization Error

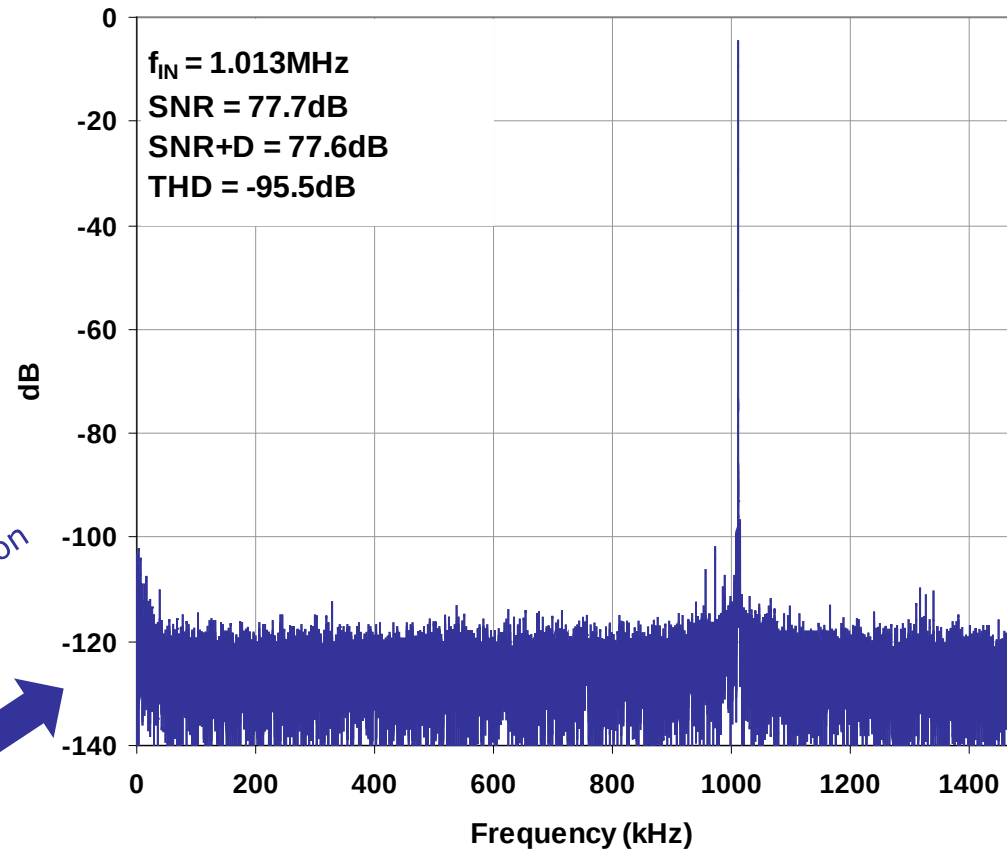


[Ref: Presentation “Data Conversion Fundamentals”, Analog Devices](#)

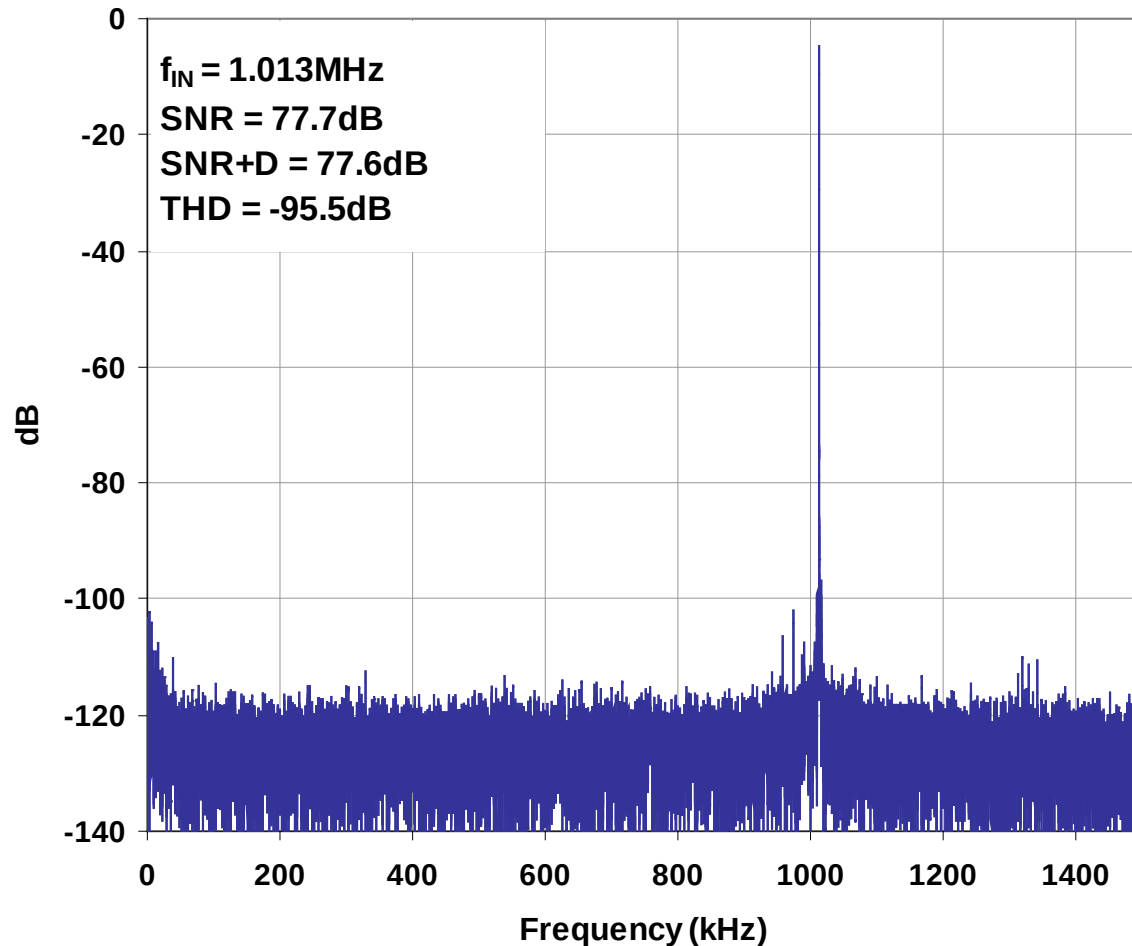
Ideal Sinewaves and their Spectra



Actual FFT Plot for AD7484, 14-Bit SAR ADC Sampling at 3MHz



[Ref: Presentation "Data Conversion Fundamentals", Analog Devices](#)



SINAD (Signal-to-Noise-and-Distortion Ratio)

The ratio of the rms signal amplitude to the noise and the mean value of the root-sum-squares (RSS) of all other spectral components, including harmonics, but excluding dc.

ENOB (Effective Number of Bits)

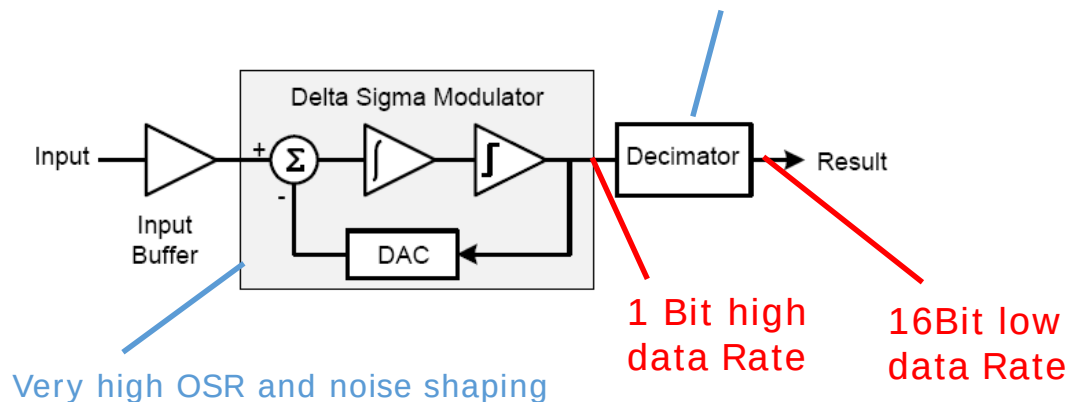
$$ENOB = \frac{SINAD - 1.76\text{dB}}{6.02}$$

The ENOB is always lower, than the hard-wired number of digital bits, because of the present of noise and distortion.
Rule of thumb: The ENOB is 1.5 or 2 Bits lower than the number of hard wired Bits.

[Ref: Presentation "Data Conversion Fundamentals", Analog Devices](#)

Principal of $\Delta\Sigma$ - ADCs

Resolution vs. Bandwidth



Properties of PSoC $\Delta\Sigma$ - ADC

8Bit to 20Bit resolution

8 sps to 384 ksp

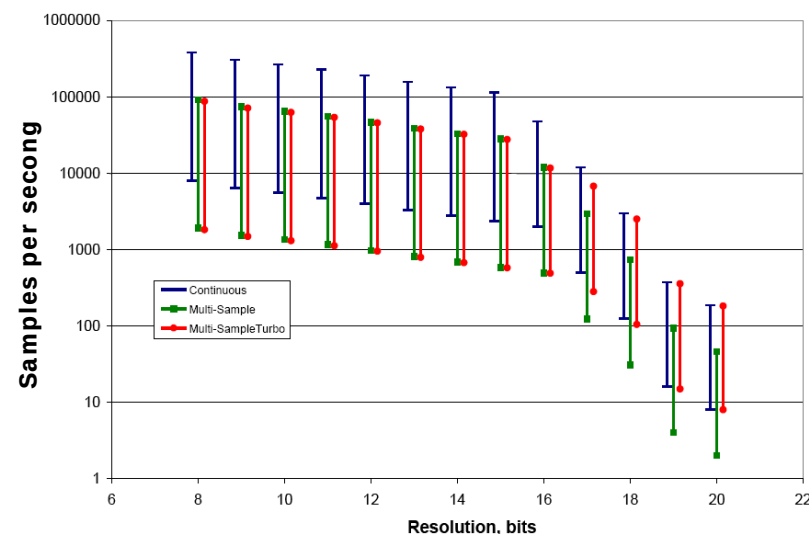
Input Buffer

4 Different Sampling modes

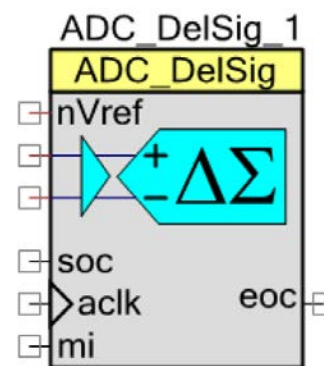
Differential input or single ended input

Hardware and software triggered start of conversion

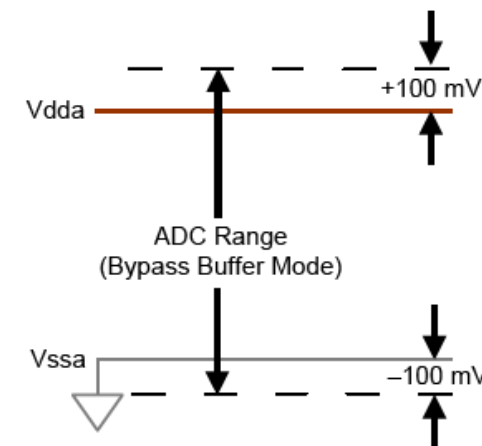
Resolution vs. Bandwidth p.7



Top Schematic View



Maximum Input Range to the ADC



Reference: PSoC Creator Component Datasheet "Delta Sigma Analog to Digital Converter Version 3.30"

The ADC_DelSig operates in one of four modes:

Mode	Description
0 - Single Sample	The ADC produces one sample per startup conversion. The interrupt should be enabled for ADC conversion with Single Sample conversion mode when the resolution is above 16 bits. To do so, enable the Global Interrupt (by calling CYGlobalIntEnable) in the application (<i>main.c</i>). Single Sample conversion mode allows queuing of one conversion. If a sample is requested either by calling the StartConvert() function or by using the "soc" input before the current conversion is complete, another conversion will automatically be started after the current conversion is complete.
1 - Multi-Sample	Multi-sample mode captures single samples back to back, resetting itself and the modulator between each sample automatically. This mode is useful when the input is switched between multiple signals. The filters are flushed between each sample so previous samples do not affect the current conversion. Note Take care when switching signals between ADC conversions. Either switch the input quickly between conversions with hardware control or stop the ADC conversion (ADC_StopConvert()) while switching the input. Then restart the ADC conversion (ADC_StartConvert()) after the new signal has been connected to the ADC. Failure to do this may result in contamination between signals in the ADC results.
2 - Continuous	Continuous sample mode operates as a normal delta-sigma converter. Use this mode when measuring a single input signal. There is a latency of three conversion times before the first conversion result is available. This is the time required to prime the internal filter. After the first result, a conversion will be available at the selected sample rate. Do not use this mode when multiple signals are multiplexed and measured with a single ADC.
3 - Multi-Sample (Turbo)	The Multi-Sample (Turbo) mode operates identically to the Multi-Sample mode for resolutions of 8 to 16 bits. For resolutions of 17 to 20 bits, the performance of this mode is about four times faster than the Multi-Sample mode. Note Take care when switching signals between ADC conversions. Either switch the input quickly between conversions with hardware control or stop the ADC conversion (ADC_StopConvert()) while switching the input. Then restart the ADC conversion (ADC_StartConvert()) after the new signal has been connected to the ADC. Failure to do this may result in contamination between signals in the ADC results.

Configure 'ADC'

Name: ADC

Config1 Common Built-in

Comment: Einfachste Config

Configuration name: 500 ADC_500

Modes

Conversion mode: 0 - Single Sample

Resolution (bits): 8

Conversion rate (SPS): 1600 Range: 1561 - 74926 SPS

Actual conv. rate (SPS): 1600

Clock frequency (kHz): 131.200

Input options - Single ended mode

Input range: Vssa to 1.024V (0.0 to Vref)

Buffer gain: 1

Buffer mode: Rail to Rail

Reference

Reference: Internal 1.024 Volts

Vref (V): 1.024

Alignment

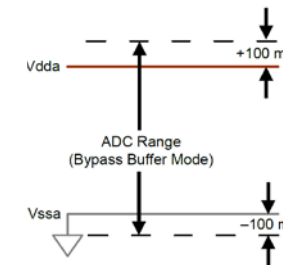
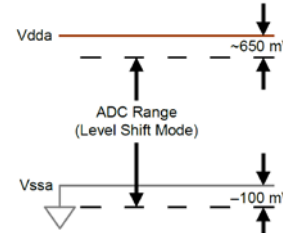
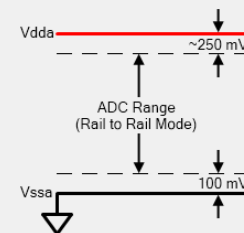
☒ Right Coherency = LOW

☐ Left 16 bits (OVF Protected)

This sets the full-scale range, which is resolved with the ADC resolution.

Here: the range of Vssa to 1.024V is mapped to 8Bit
 $\rightarrow V_{lsb} = 1.024V / 256 = 4mV$

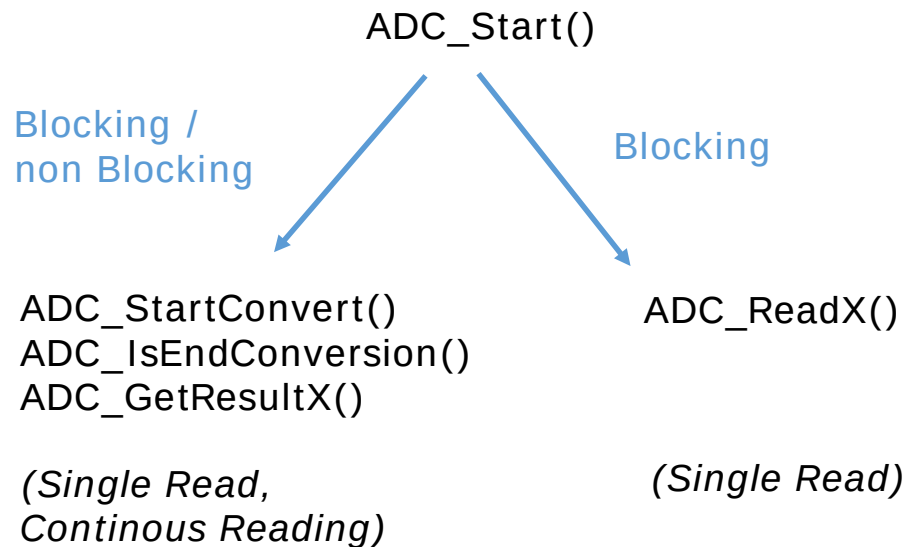
The Buffer mode influences the ADC – input range. This is the maximum input voltage per input pin. This is especially important with a differential input by determining the allowed common mode.



General Workflow for Hardware Blocks

1. Get the SD-ADC symbol to your top Schematic
2. Configure the SD-ADC to your specification
3. Connect the input and output pins
4. Compile → automated APIs are generated

Especially for SD-ADC Block

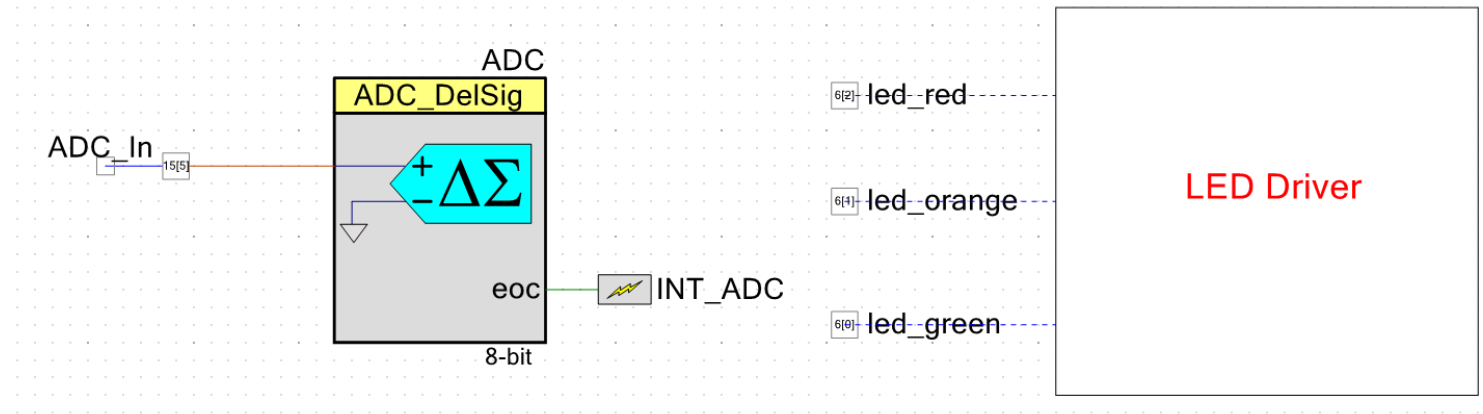


SD-API

Function	Description
<code>ADC_Start()</code>	Sets the initVar variable, calls the <code>ADC_Init()</code> function, and then calls the <code>ADC_Enable()</code> function.
<code>ADC_Stop()</code>	Stops ADC conversions and powers down
<code>ADC_SetBufferGain()</code>	Selects input buffer gain (1,2,4,8)
<code>ADC_StartConvert()</code>	Starts conversion
<code>ADC_StopConvert()</code>	Stops conversions
<code>ADC_IRQ_Enable()</code>	Enables interrupts at the end of conversion
<code>ADC_IRQ_Disable()</code>	Disables interrupts
<code>ADC_IsEndConversion()</code>	Returns a nonzero value if conversion is complete
<code>ADC_GetResult8()</code>	Returns an 8-bit conversion result
<code>ADC_GetResult16()</code>	Returns a 16-bit conversion result
<code>ADC_GetResult32()</code>	Returns a 32-bit conversion result
<code>ADC_Read8()</code>	Starts ADC conversions, waits for the conversion to be complete, stops ADC conversion and returns the signed 8-bit value of result.
<code>ADC_Read16()</code>	Starts ADC conversions, waits for the conversion to be complete, stops ADC conversion and returns the signed 16-bit value of result.
<code>ADC_Read32()</code>	Starts ADC conversions, waits for the conversion to be complete, stops ADC conversion and returns the signed 32-bit value of result.

SD-Settings

Single Sample
Single Ended
8-Bit



Calculations

$$V_{\text{LSB}} = (V_{\text{dda}} - V_{\text{ssa}}) / 2^8 = 19.53 \text{ mV}$$

Input Voltage	ADC Measurements		LED
(V) <i>Voltmeter</i>	Digits	Corresponding Volts	
			Green
			Yellow / Orange
			Red

Exercise 1: ADC Reading

Aim

Use a 16-Bit SD-ADC to read the voltage of a voltage divider providing 3 different voltages: $0.088 \cdot V_{DDA}$, $0.5 \cdot V_{DDA}$ and $0.911 \cdot V_{DDA}$. Depending on which voltage the ADC reads, the green, orange / yellow or red LED is turned on.

Among other steps, do the following

- Design a voltage divider providing you the necessary voltages for the test (use the resistors of your lab box)
- Fill in a table of voltage measured by instrument / ADC samples / equivalent voltage by ADC samples (handcalculated) / use the function `ADC_CountsTo_mVolts` and compare.

Hint:

Use the debugger to get the adc readings.

- Use these Pin Mappings

☒	ADC_In	P15[5]	▼	94	▼	☒
☒	DAC_out	P15[4]	▼	93	▼	☒
☒	led_green	P2[5]	▼	1	▼	☒
☒	led_orange	P2[6]	▼	2	▼	☒
☒	led_red	P2[7]	▼	3	▼	☒

Name: ADC

Config1 Common Built-in

Comment: Einfachste Config

Configuration name: 500 ADC_500

Modes

Conversion mode: 0 - Single Sample ▼

Resolution (bits): 8 ▼

Conversion rate (SPS): 1600 ▼ Range: 1561 - 26829 SPS

Actual conv. rate (SPS): 1600

Clock frequency (kHz): 131.200

Input options - Single ended mode

Input range: Vssa to Vdda ▼

Buffer gain: 1 ▼

Buffer mode: Bypass Buffer ▼

Reference

Reference: Internal Vdda/4 ▼

Vref (V): 1.250 ▼

Alignment

☒ Right Coherency = LOW

☐ Left 16 bits (OVF Protected) ▼

Hints for Your Project

Input range

Vdda

+100 mV

ADC Range (Bypass Buffer Mode)

Vssa

-100 mV

Literature

- “*Delta Sigma Analog to Digital Converter (ADC_DelSig)*” version 3.30 , PSoC® Creator™ Component Datasheet (pdf)
- [“Data Conversion Fundamentals”, Analog Devices](#), Presentation pptx
- “The Data Conversion Handbook”, Analog Devices, Editor Walt Kester